

Course Projects

ECE 489 / ME 446 — Robot Dynamics and Control
Spring 2026

General Information

- **Team size:** 2–3 students per team.
- **Choose one** of the two projects below.
- **Simulation requirement:** Every project **must** include experiments conducted in a physics simulator—**MuJoCo**, **NVIDIA Isaac Gym / Isaac Lab**, or **Gazebo**. Teams that additionally demonstrate results on a **real robot** will receive bonus credit.
- **Timeline & Grading:**

Phase	Deliverables	Weight
Week 6 — Proposal	2-page proposal: team members, chosen project & approach, simulation platform, robot model, planned milestones & task division.	5%
Week 10 — Mid-term Report	4–5 page progress report: simulation environment setup (screenshots), preliminary results, theoretical derivation or algorithm formulation in progress, difficulties & revised plan.	10%
Week 14 — Presentation & Demo	12-min in-class presentation + recorded demo video (3–5 min) + live Q&A. Show simulation results on at least two conditions.	20%
Week 16 — Final Report & Code	10–15 page final report + code repository. Report must include: theoretical derivation / algorithm design (20%), simulation implementation & results (30%), quantitative analysis & comparison of methods (15%).	65%
Bonus — Real Robot	Demonstrate results on a real robot; document sim-to-real gap.	+20%

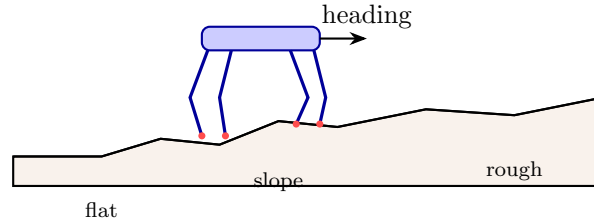
- **Recommended platforms & getting started:**
 - **MuJoCo** (<https://mujoco.org>): free, fast contact dynamics, Python bindings via `mujoco` or `dm_control`. Start with the official tutorial: <https://mujoco.readthedocs.io>.
 - **NVIDIA Isaac Gym / Isaac Lab** (<https://developer.nvidia.com/isaac>): GPU-parallel simulation; best for RL training at scale.
 - **Gazebo** (<https://gazebo.org>): ROS-integrated, good for teams planning real-robot deployment.

- **Robot models (URDF/MJCF):** https://github.com/google-deepmind/mujoco_menagerie — ready-to-use models for Franka Panda, UR5e, Unitree Go1/Go2, Kuka iiwa, Shadow Hand, and many more.

Project 1: Quadruped Locomotion — Model-Based Control and Learning-Based Policies

Objective

Develop locomotion controllers for a quadruped robot (e.g., Unitree Go1/A1/Go2, AnyMal) in a physics simulator. You will implement **at least one** of the two approaches below (model-based *or* learning-based) and compare it against a provided or reproduced baseline.



Quadruped traversing terrain with varying elevation.

Tasks

1.1 Simulation Environment Setup.

- Load a quadruped model (Unitree Go1 or A1 recommended). Models are directly available in `mujoco_menagerie` and in the Isaac Gym / Isaac Lab asset repos. Verify joint ranges, torque limits, and total mass against the robot's datasheet.
- Create **at least two terrain types**: (i) flat ground, and (ii) one challenging terrain of your choice (slope $\sim 10\text{--}15^\circ$, random bumps, or stairs). For Isaac Lab users, use the built-in terrain generator; for MuJoCo, use `hfield` assets.
- Log data at each time-step: center-of-mass (CoM) position and velocity, foot contact forces, joint torques, body orientation (roll/pitch/yaw).

1.2 Approach A — Model-Based: Gait Generation + Force Control.

- Single Rigid Body Dynamics (SRBD):** Model the torso as a single rigid body with ground reaction forces $\mathbf{f}_i$ at the four feet:

$$m \ddot{\mathbf{p}} = \sum_{i=1}^4 \mathbf{f}_i + m \mathbf{g}, \quad \mathbf{I} \dot{\boldsymbol{\omega}} + \boldsymbol{\omega} \times \mathbf{I} \boldsymbol{\omega} = \sum_{i=1}^4 (\mathbf{r}_i - \mathbf{p}) \times \mathbf{f}_i.$$

- Gait scheduler:** Implement at least one periodic gait—**trot** (diagonal leg pairs move together). Define contact/swing timing via a phase clock. Generate foot swing trajectories using parabolic, cubic-spline, or Bézier curves with configurable step height and stride length.

- (c) **Stance-phase force control:** Solve a QP (quadratic program) at each control step to find ground reaction forces $\mathbf{f}_i$ that track a desired CoM acceleration and body orientation while satisfying friction-cone constraints ($\|\mathbf{f}_{i,\text{tan}}\| \leq \mu f_{i,\text{norm}}$). You may use `qpOASES`, `osqp`, or `scipy.optimize`.
- (d) Map foot forces to joint torques via the leg Jacobian: $\boldsymbol{\tau}_{\text{leg}} = -J_{\text{leg}}^T \mathbf{f}_i$ (stance); use PD tracking for swing legs.
- (e) **Demo:** Trot on flat ground at $\geq 0.4\text{m/s}$ for $\geq 10\text{s}$ without falling. Plot CoM height, body roll/pitch, and a gait diagram (Gantt chart of foot contacts).

Tip: The open-source repos listed in the References section provide excellent starting points for the QP formulation and gait timing. You are allowed to study and adapt their structure, but your code and report must clearly describe what you implemented vs. what you borrowed.

1.3 Approach B — Learning-Based: RL Locomotion Policy.

- (a) Train a locomotion policy using **PPO** (Proximal Policy Optimization). Observation: body orientation, angular velocity, joint positions and velocities, velocity command. Action: desired joint positions (fed to a low-level PD controller at each joint).
- (b) Design a reward that encourages: (i) forward velocity tracking, (ii) upright body orientation, (iii) smooth joint motions (penalize jerk and torque), (iv) adequate foot clearance during swing.
- (c) Apply **domain randomization** during training: vary friction ($\mu \in [0.5, 1.2]$), payload mass ($\pm 20\%$), and motor strength ($\pm 10\%$). If using Isaac Gym / Isaac Lab, train with ≥ 512 parallel environments.
- (d) Report training curves (reward vs. steps) and the final policy’s walking behavior.

Tip: Start from the `legged_gym` codebase (see References). Modify the reward terms and environment parameters for your chosen robot.

1.4 Evaluation & Comparison.

- (a) If you implemented only one approach, compare against a simple **baseline**: open-loop sinusoidal joint trajectories (“CPG-like” baseline) with hand-tuned amplitude and frequency. If you implemented both, compare them directly.
- (b) Test on flat ground *and* your challenging terrain at two commanded speeds. Report (over ≥ 10 trials each):
 - Velocity tracking error (RMS).
 - Body stability: RMS roll & pitch.
 - Energy: Cost of Transport, $\text{CoT} = \frac{\sum |\tau_i \dot{q}_i| \Delta t}{m g d}$.
 - Robustness: apply a lateral push (30–50 N for 0.1 s) and record recovery success.
- (c) Discuss: strengths and failure modes of your approach vs. the baseline.

1.5 Extension (optional, for bonus credit — choose any).

- (a) **MPC:** Replace the single-step QP with a receding-horizon Model Predictive Controller using the SRBD model (see `Quadruped-PyMPC` in References).
- (b) **Additional gaits:** Implement *walk*, *pace*, or *bound* and show gait transitions.
- (c) **Sim-to-real:** Deploy a policy on a real Unitree robot and document the sim-to-real gap.

Deliverables

- Code repository: simulation scene, controller(s), evaluation scripts.
- If RL: trained policy checkpoint and training curves.
- Demo video: walking on flat ground + challenging terrain.
- Report: dynamics derivation or RL formulation, controller/policy design, quantitative evaluation with plots, and discussion.

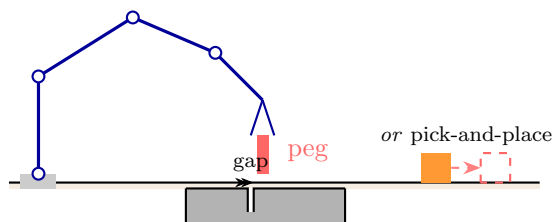
References & Starter Code

- [MPC theory] J. Di Carlo et al., “Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control,” *IROS*, 2018.
 - [RL training] J. Rudin et al., “Learning to Walk in Minutes Using Massively Parallel Deep RL,” *CoRL*, 2022.
 - [Textbook] K. M. Lynch & F. C. Park, *Modern Robotics*, Cambridge, 2017, Ch. 8 (dynamics of open chains).
 - [Robot models] https://github.com/google-deepmind/mujoco_menagerie — Unitree Go1, Go2, A1 MJCF models.
 - [RL framework] https://github.com/leggedrobotics/legged_gym — Isaac Gym environments for legged robots (PPO training, terrain generation, reward design).
 - [MPC code (Python)] <https://github.com/iit-DLSLab/Quadruped-PyMPC> — SRBD-based MPC for quadrupeds using `acados` or `jax`.
 - [MPC code (Python)] <https://github.com/yinghansun/pympc-quadruped> — Convex MPC implementation following the MIT Cheetah approach.
 - [Isaac Lab] <https://github.com/isaac-sim/IsaacLab> — NVIDIA’s latest simulation framework (successor to Isaac Gym).
 - [Sim-to-real RL] https://github.com/yxyang/fast_and_efficient — Efficient sim-to-real locomotion policy training.
-

Project 2: Robotic Manipulation — Impedance Control and Imitation Learning

Objective

Develop a controller for a robot arm to perform a **contact-rich manipulation task** (e.g., peg-in-hole insertion, pick-and-place, or surface wiping) in a physics simulator. You will implement **at least one** of the two approaches below—**impedance control** (model-based) *or* **imitation learning** (data-driven)—and compare it with a baseline.



Example tasks: peg-in-hole insertion (left) or pick-and-place (right).

Tasks

2.1 Simulation Environment Setup.

- (a) Load a 6- or 7-DOF robot arm with a gripper in your chosen simulator (Franka Panda or UR5e recommended; both have ready-to-use MJCF/URDF files). Verify joint limits and torque limits.
- (b) Build a task scene. Choose **one** task:
 - **Peg-in-hole:** cylindrical peg + fixture with a hole; clearance $\leq 5\%$ of peg diameter.
 - **Pick-and-place:** 2–3 objects on a table; robot must grasp an object and place it at a goal pose.
 - **Surface wiping / polishing:** end-effector must maintain contact with a surface while following a desired path and applying a target normal force.

If you choose peg-in-hole, also implement a wrist **force/torque sensor** (read the 6-axis wrench from the simulator’s contact API). For pick-and-place, ensure the objects have realistic mass and friction.

- (c) Implement **forward kinematics** (PoE or DH) and the **Jacobian** for your robot. Validate against the simulator’s built-in FK for ≥ 10 random configurations.

2.2 Approach A — Model-Based: Impedance / Force Control.

- (a) Derive a **task-space impedance controller**. Starting from the manipulator equation $M(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau} + J^T \mathbf{f}_{\text{ext}}$, command joint torques so the end-effector behaves as a mass–spring–damper:

$$M_d \ddot{\tilde{\mathbf{x}}} + D_d \dot{\tilde{\mathbf{x}}} + K_d \tilde{\mathbf{x}} = \mathbf{f}_{\text{ext}}, \quad \tilde{\mathbf{x}} = \mathbf{x} - \mathbf{x}_d.$$

Explain the role of each parameter (M_d, D_d, K_d) and the difference between impedance and admittance control.

- (b) Design a **task-specific strategy** using your impedance controller:
 - *Peg-in-hole:* Multi-phase approach $\rightarrow$ search/align (spiral or wobble) $\rightarrow$ compliant insertion.
 - *Pick-and-place:* Waypoint-based Cartesian trajectory (approach $\rightarrow$ grasp $\rightarrow$ lift $\rightarrow$ transport $\rightarrow$ place $\rightarrow$ retract) with impedance-controlled execution.
 - *Wiping:* Track a desired end-effector path on the surface with force regulation in the normal direction.
- (c) **Experiments:** Vary a task-relevant parameter (e.g., initial offset for peg-in-hole, object position for pick-and-place, desired normal force for wiping). Run ≥ 10 trials per condition. Report success rate, task completion time, and peak contact force. Compare with a **stiff position-control baseline** (high-gain PD, no compliance) and show that impedance control is safer or more successful.

2.3 Approach B — Learning-Based: Imitation Learning / RL.

- (a) **Data collection:** Collect 30–50 demonstrations of the task. You may use:
 - Scripted demonstrations (e.g., generate trajectories with your impedance controller from Approach A or a hand-coded policy).
 - Teleoperation in the simulator (MuJoCo and **robosuite** support keyboard/spacemouse teleop).

Store observations (joint positions, end-effector pose, and optionally wrist F/T) and actions (joint position or velocity targets).

- (b) **Policy training:** Train a policy using one of:
- **Behavior Cloning (BC):** A feedforward neural network mapping observations to actions.
 - **ACT (Action Chunking with Transformers):** Predicts a *chunk* of future actions; available in the LeRobot framework (see References).
 - **RL fine-tuning:** Use the BC policy as initialization, then fine-tune with PPO or SAC in the simulator.

Report training loss curves and qualitative behavior at different training stages.

- (c) **Experiments:** Evaluate the learned policy on ≥ 20 trials with randomized initial conditions (e.g., object pose ± 1 cm, peg offset ± 2 mm). Report success rate and compare with the scripted/impedance baseline.

2.4 Analysis.

- (a) If you implemented both approaches, compare them directly. If you implemented only one, compare with the stiff-PD baseline and discuss how the other approach would differ.
- (b) Discuss: (i) how much task-specific engineering is required for each approach; (ii) how each approach handles uncertainty; (iii) what would be needed to transfer your controller to a real robot.

2.5 Extension (optional, for bonus credit — choose any).

- (a) **Variable impedance:** Automatically switch impedance parameters based on task phase (free-space $\rightarrow$ contact $\rightarrow$ insertion) using a state machine or learned schedule.
- (b) **Vision-based policy:** Add a simulated camera and train a visuomotor policy (image observations $\rightarrow$ actions) using LeRobot or robomimic.
- (c) **Sim-to-real:** Deploy on a real robot arm. Document calibration steps, sensor noise, and the sim-to-real performance gap.

Deliverables

- Code repository: simulation scene, controller(s), data collection / training scripts, evaluation scripts.
- If learning-based: trained policy checkpoint, demonstration dataset, and training curves.
- Demo video: task execution under nominal and perturbed conditions.
- Report: controller derivation or learning formulation, task strategy, quantitative evaluation, and discussion.

References & Starter Code

- **[Impedance control]** N. Hogan, “Impedance Control: An Approach to Manipulation,” *ASME J. Dyn. Sys.*, 107(1), 1985.
- **[Operational space]** O. Khatib, “A Unified Approach for Motion and Force Control of Robot Manipulators,” *IEEE J. Robotics and Automation*, 3(1), 1987.
- **[Textbook]** B. Siciliano et al., *Robotics: Modelling, Planning and Control*, Springer, 2009, Ch. 8–9.

- [Textbook] K. M. Lynch & F. C. Park, *Modern Robotics*, Cambridge, 2017, Ch. 4–6, 11.
 - [LeRobot] <https://github.com/huggingface/lerobot> — Hugging Face’s end-to-end robot learning framework; includes **ACT**, diffusion policy, BC, and data collection tools. Supports MuJoCo and real robots. Excellent starting point for imitation learning.
 - [robosuite] <https://github.com/ARISE-Initiative/robosuite> — MuJoCo-based manipulation benchmark with Franka Panda, UR5e, peg-in-hole, pick-and-place, and wiping tasks. Includes teleoperation support.
 - [robomimic] <https://github.com/ARISE-Initiative/robomimic> — Learning-from-demonstration framework; BC, BC-RNN, and more. Works directly with robosuite datasets.
 - [Robot models] https://github.com/google-deepmind/mujoco_menagerie — Franka Panda, UR5e, Kuka iiwa MJCF models.
 - [MuJoCo tutorial] <https://mujoco.readthedocs.io/en/stable/python.html> — Official Python bindings documentation.
 - [Isaac Lab manipulation] <https://github.com/isaac-sim/IsaacLab> — GPU-parallel environments for manipulation and locomotion.
-

Project Summary

#	Title	Core Topics	Rec. Simulator
1	Quadruped Locomotion	SRBD, gait generation, QP force control and/or RL policy (PPO), terrain traversal, domain randomization	Isaac Gym / MuJoCo
2	Robotic Manipulation	FK/Jacobian, impedance control and/or imitation learning (ACT / BC), contact-rich tasks (peg-in-hole, pick-and-place)	MuJoCo / Isaac Lab

Note: Each project requires implementing **at least one** approach (model-based or learning-based); implementing both will be viewed favorably. You may build upon the open-source repos listed in the references—clearly credit any code you reuse and describe your own contributions. Creative extensions, thorough analysis, and real-robot demonstrations will be rewarded.